Pontifícia Universidade Católica do Rio Grande do Sul
Fundamentos de Processamento Paralelo e Distribuído
Profª Eduarda Rodrigues Monteiro

Trabalho 1 – Concorrência e Sincronização: O Problema dos Filósofos Jantando

Diogo Luiz Zago
Enrico Canabarro Lazzari
Yan Pantoja

Outubro de 2026

Trabalho 1 — Concorrência e Sincronização: O Problema dos Filósofos Jantando

1. Soluções implementadas e mecanismos de sincronização

Todas as versões usam 5 filósofos, 20 ciclos por filósofo (pensar → adquirir garfos → comer → liberar garfos), 2 s pensando e 3 s comendo. Os experimentos foram realizados na máquina descrita na Seção 5; os resultados completos estão em `results/todas.csv`.

V0 — Sequencial. Os filósofos executam um após o outro, sem goroutines. Não há disputa por garfos, portanto não há mecanismo de sincronização. O tempo total é determinístico: $5 \times 20 \times (2+3)$ s $= 500$ s, confirmado pelos dados (média 500,30 s).

V1 — Concorrente sujeita a deadlock. Uma goroutine por filósofo; garfos como `[5]sync.Mutex`. Cada filósofo trava o garfo esquerdo e depois o direito. Uma goroutine `vigia()` encerra o processo se não houver progresso em 10 s, evidenciando o deadlock.

V2 — Sem deadlock, com sincronização excessiva. Igual à V1, acrescida de um semáforo — implementado como canal com buffer de capacidade 4 (`make(chan struct{}, 4)`) — que cada filósofo adquire antes de tentar pegar os garfos e libera após devolvê-los. No máximo 4 filósofos disputam garfos ao mesmo tempo.

V3 — Concorrente aprimorada. Implementa duas estratégias selecionáveis via flag `-strategy`:

- **Monitor** (padrão): cada filósofo possui um estado (PENSANDO, FAMINTO, COMENDO) protegido por um `sync.Mutex` de monitor e uma variável de condição `sync.Cond` própria. `pegarGarfos(i)` marca o filósofo como FAMINTO e chama `testar(i)`, que só o promove a COMENDO se nenhum vizinho estiver comendo — caso contrário, ele dorme em `cond[i].Wait()`. `soltarGarfos(i)` marca PENSANDO e acorda os vizinhos famintos. Os dois garfos são obtidos atomicamente: nenhum filósofo segura um enquanto espera o outro.
- **Ordenada** (`ordered`): garfos como `sync.Mutex`; cada filósofo trava primeiro o garfo de menor índice entre os seus dois. O filósofo 4, em vez de tentar o garfo 4 e depois o 0, tenta o 0 primeiro — quebrando o ciclo de espera.

Análise comparativa das estratégias de sincronização

	Mecanismo principal	Condição de Coffman quebrada	Máx. comendo (teórico)
V0	nenhum (sequencial)	—	1
V1	`sync.Mutex` por garfo	nenhuma → deadlock	2
V2	mutex + semáforo (canal buffer 4)	espera circular	2
V3 monitor	`sync.Mutex` + `sync.Cond`	posse-e-espera	2
V3 ordered	mutex por garfo, aquisição ordenada	espera circular	2

A V2 e a V3 são ambas corretas, mas quebram condições distintas. A V2 ainda permite que um filósofo segure o garfo esquerdo enquanto espera o direito; com tempos iguais os quatro admitidos formam uma corrente de espera em que só o da ponta come — produzindo execução quase serializada. A V3 elimina a posse-e-espera (monitor) ou o ciclo de recursos (ordered), permitindo que não-vizinhos comam em paralelo.

2. Resultados experimentais

Tabela comparativa

Solução	Estratégia	Exec.	Tempo médio (s)	Tempo máx. (s)	Espera média (s)	Maior espera (s)	Máx. comendo	Ciclos/filósofo
V0	sequencial	5	500,32	500,34	0,000	0,000	1	20
V1	—	1	—	—	—	—	0	0
			(deadlock)					
V2	semaforo4	5	302,09	302,10	9,802	12,007	1	20
V3	monitor	5	158,05	176,09	2,420	45,027	2	20
V3	ordered	5	174,24	182,04	2,294	9,004	2	20

Médias e máximos calculados sobre as execuções de cada solução. “Maior espera” é o maior valor observado em qualquer execução. V1 não conclui; a linha registra o comportamento observado (deadlock após ciclo 0).

Gráficos comparativos

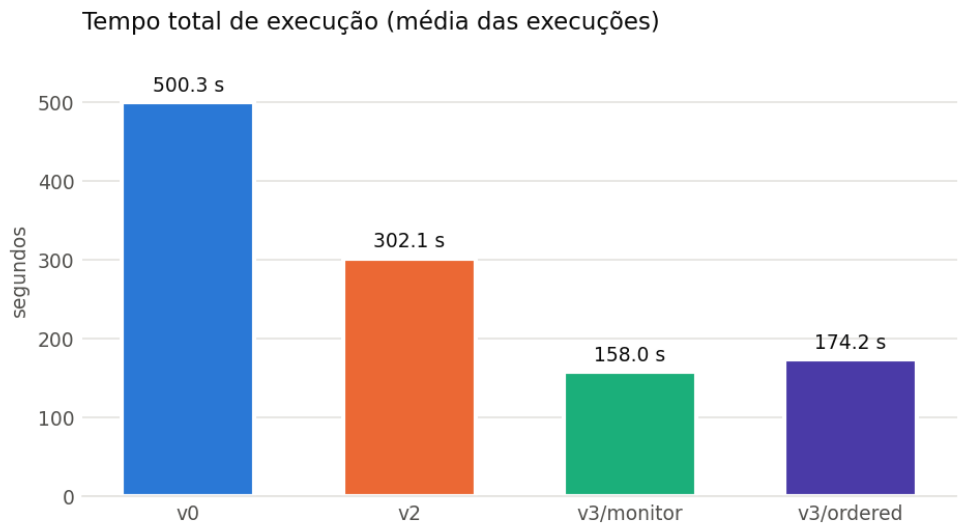


Figure 1: Tempo total de execução por solução (médias sobre 5 execuções).

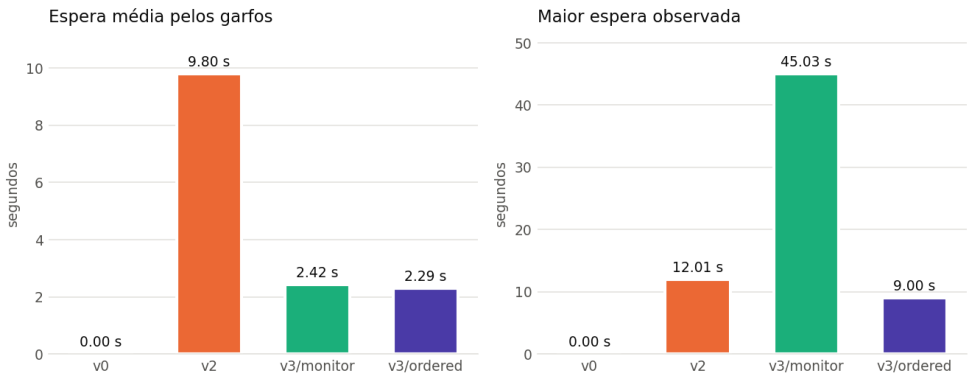


Figure 2: Tempo médio de espera pela aquisição dos garfos.

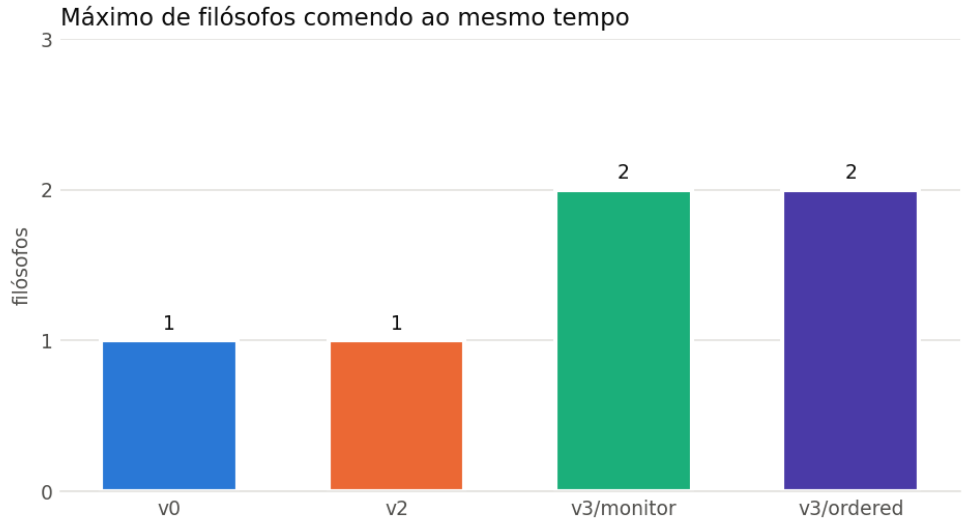


Figure 3: Número máximo de filósofos comendo simultaneamente.

Evidência do deadlock da V1

A saída completa está em `results/v1_deadlock.txt`. Os cinco filósofos pensaram (T1), cada um travou o garfo esquerdo (L1), e nenhum chegou a comer (E1). O `vigia()` detectou 10 s sem progresso e encerrou o processo:

```
[P1] [P2] [P3] [P4] [P5]

                    T1
 T1
      T1
           T1
                T1
                L1
           L1
      L1
 L1
                    L1

DEADLOCK detected! No progress in the last 10s
Cycles per philosopher: [0 0 0 0 0]
```

Cada filósofo i segura o garfo i e aguarda o garfo $(i+1)$ mod 5, formando o ciclo $P_0 \to P_1 \to P_2 \to$

$P_3 \to P_4 \to P_0$. As quatro condições de Coffman estão satisfeitas: exclusão mútua (cada garfo é um mutex), posse-e-espera (segura o esquerdo aguardando o direito), ausência de preempção (ninguém retira o garfo de outro) e espera circular (o ciclo descrito acima).

3. Conclusão e análise dos resultados

1. Como funciona cada solução?

- **V0:** filósofos executam em sequência — um termina todos os 20 ciclos antes do próximo começar.
- **V1:** cada filósofo é uma goroutine que tenta travar o garfo esquerdo e depois o direito; sem proteção contra deadlock.
- **V2:** igual à V1, mas um semáforo de 4 vagas impede que todos os 5 tentem pegar garfos ao mesmo tempo.
- **V3 (monitor):** filósofo só "possui" garfos quando obtém os dois juntos, dentro do monitor; enquanto FAMINTO, não segura nenhum e dorme em sua variável de condição.
- **V3 (ordered):** filósofo trava sempre o garfo de menor índice primeiro, quebrando o ciclo de dependências.

2. Quais mecanismos de sincronização são utilizados?

- **Exclusão mútua:** `sync.Mutex` por garfo (V1, V2, V3-ordered) e mutex do monitor (V3-monitor).
- **Semáforo contador:** V2 — canal com buffer (`make(chan struct{}, 4)`).
- **Variáveis de condição:** V3-monitor — `sync.Cond` por filósofo.
- **Sincronização de término:** `sync.WaitGroup` em todas as versões concorrentes.
- **Proteção das métricas:** mutex próprio `statsMu` em todas as versões concorrentes.

3. Como esses mecanismos são implementados em Go?

Go não possui semáforo nem variável de condição como primitivas da linguagem; ambos são emulados:

- **Mutex:** `sync.Mutex` com `Lock()`/`Unlock()`. É não-reentrante.
- **Semáforo:** canal com buffer — `sem <- struct{}{}` bloqueia ao atingir a capacidade; `<-sem` libera uma vaga. A semântica de canal garante FIFO entre goroutines bloqueadas.
- **Variável de condição:** `sync.Cond`, criada com `sync.NewCond(&mu)`. `Wait()` libera atomicamente o mutex e suspende a goroutine; ao ser acordada por `Signal()`, retrava o mutex antes de retornar. O padrão correto exige re-checar a condição num laço `for state[i] != comendo { cond[i].Wait() }`.
- **Concorrência:** cada filósofo é uma goroutine (`go philosopher(...)`); o `main` aguarda todas com `wg.Wait()`.

4. Por que ocorre deadlock na V1?

As quatro condições de Coffman estão simultaneamente satisfeitas. Com tempos de pensar iguais todos os filósofos terminam de pensar juntos, travam o garfo esquerdo e ficam aguardando o direito — que está com o vizinho. O `time.Sleep(1 ms)` entre os dois `Lock` amplia a janela de race e torna o deadlock praticamente certo a cada execução: a evidência registrada mostra os 5 filósofos em L1 e 0 ciclos concluídos.

5. Como V2 e V3 evitam deadlock?

- **V2** quebra a **espera circular**: com no máximo 4 filósofos disputando 5 garfos, pelo princípio da casa dos pombos pelo menos um sempre consegue os dois, come e libera.
- **V3-monitor** quebra a **posse-e-espera**: o filósofo nunca segura um garfo esperando o outro — ele só passa a COMENDO quando obtém os dois atomicamente dentro do monitor.
- **V3-ordered** quebra a **espera circular**: a ordenação global de recursos impede que o ciclo $P_0 \to \cdots \to P_4 \to P_0$ se forme, pois P_4 disputa o garfo 0 (não o 4) com P_0.

6. Existe possibilidade de starvation? Justifique.

- **V3-monitor:** sim, em teoria. Se os dois vizinhos de P_i se alternam comendo sem ambos estarem livres ao mesmo tempo, `testar(i)` nunca dispara. O algoritmo não mantém fila de espera. Na prática, com tempos iguais para todos, isso é improvável — e os dados confirmam: todos os filósofos completaram 20 ciclos em todas as execuções.
- **V2 e V3-ordered:** dependem de `sync.Mutex`, que não garante ordem FIFO por padrão (Go ativa um modo anti-starvation após ~1 ms de espera, mas não há garantia absoluta por construção).
- **V2:** o canal acorda goroutines bloqueadas em ordem FIFO, o que reduz o risco no semáforo em si, mas o mutex dos garfos ainda não é justo.

7. Qual solução apresentou maior tempo total de execução?

Entre as que concluem normalmente, a **V0** com 500,32 s em média — o valor teórico exato de $5 \times 20 \times 5$ s. A V2 ficou em 302,09 s, e a V3 entre 158 s e 174 s dependendo da estratégia.

8. Qual solução apresentou maior tempo de espera pelos recursos?

A **V2**, com espera média de **9,802 s** e maior espera observada de **12,007 s**. A V3-monitor teve espera média de 2,420 s e a V3-ordered, 2,294 s — aproximadamente 4× menor que a V2. A V0 tem espera zero por definição (sem concorrência).

9. Quantos filósofos conseguiram comer simultaneamente em cada solução?

- **V0:** 1, por construção (sequencial).
- **V1:** 0 ciclos concluídos; deadlock antes de qualquer refeição.
- **V2:** máximo de **1** em todas as 5 execuções.
- **V3-monitor:** máximo de **2** em todas as 5 execuções.
- **V3-ordered:** máximo de **2** em todas as 5 execuções.

O máximo teórico é 2: com 5 filósofos em círculo e cada um usando 2 garfos adjacentes, $\lfloor 5/2 \rfloor = 2$ é o limite geométrico.

10. Qual foi o impacto da sincronização excessiva da V2?

A V2 levou 302,09 s contra 158,05 s da V3-monitor — **1,91× mais lenta**, com max_comendo = 1 em todas as execuções e espera média de 9,802 s (vs. 2,420 s da V3). O semáforo garante ausência de deadlock, mas não elimina a corrida pelo garfo direito: os quatro filósofos admitidos travam o esquerdo quase simultaneamente e formam uma corrente de espera em que só o da ponta come. A execução fica praticamente serializada, mesmo com 14 núcleos disponíveis. O resultado é pior que o sequencial em tempo de espera, mas melhor que o sequencial em tempo total porque o pensar (2 s) ocorre em paralelo.

11. A solução com maior sincronização foi necessariamente a de melhor desempenho?

Não. A V2 acrescenta um semáforo sobre os mesmos mutexes da V1 e apresenta o pior desempenho entre as versões concorrentes. A V3 também usa múltiplas primitivas (monitor + variáveis de condição), mas sincroniza **a decisão** de comer, não o ato de comer — a trava do monitor fica ocupada por microssegundos, e os 3 s de refeição ocorrem fora dela, em paralelo entre não-vizinhos.

O que determina o desempenho não é a quantidade de primitivas, mas **quanto do trabalho real fica dentro da região serializada**. Na V2, o ato de adquirir garfos é serializado pelo semáforo; na V3, apenas a verificação de estado é serializada.

12. Qual é a relação entre o grau de concorrência e o tempo total de execução?

É inversa. O trabalho total de comer é fixo ($5 \times 20 \times 3$ s $= 300$ s); com no máximo k filósofos comendo ao mesmo tempo, o tempo fica limitado por baixo em $\approx 300/k$ s:

- $k = 1$ (V0 e, na prática, V2): tempo entre 300 s e 500 s — V0: 500 s, V2: 302 s.
- $k = 2$ (V3): tempo cai para ≈ 150 s — V3-monitor: 158 s, V3-ordered: 174 s.

A diferença entre V2 e V3 ilustra que aumentar o grau de concorrência de 1 para 2 reduziu o tempo em ~47%. A relação não é linear porque o tempo de pensar (2 s) também se sobrepõe às refeições dos outros, mas a tendência é clara: maior concorrência real → menor tempo total.

---

4. Link do repositório

Código-fonte completo em: `https://github.com/enricolazzari/t1-FPPD`

---

5. Ambiente de execução

```
Linguagem:    Go 1.26.5
SO:           Linux 6.18.33.2-microsoft-standard-WSL2 x86_64 (linux/amd64)
Processador:  Intel(R) Core(TM) Ultra 5 135U
Cores:        14
```

Compilação e execução:

```
go build -o build/v0 v0/v0.go
go build -o build/v1 v1/v1.go
go build -o build/v2 v2/v2.go
go build -o build/v3 v3/v3.go

# Rodar todos os experimentos (5 execuções por versão):
bash scripts/run_experiments.sh

# Gerar tabela e gráficos:
python3 scripts/report.py
```